Nama: Muhammad Risza Dhinas Anggara
NIM : 2315168072/TK/56872
Assignment 03 - Written Analysis

Problem 1: Algorithm Analysis

a) Snippet A

```
//n >= 1
int sum = 0;
for (int i=0; i<n; i++){
    for (int j=0; j<i; j++){
        sum += arr[i]+arr[j];
    }
}
```

- int sum = 0 → 1
- int i = 0 → 1
- i<n → n+1
- i++ → n
- int j = 0 → n
- j<i → n(n+1)/2
- j++ → n(n-1)/2
- arr[i] access → n(n-1)/2
- arr[j] access → n(n-1)/2
- arr[i] + arr[j] → n(n-1)/2
- sum += → n(n-1)/2

(i) $T(n) = 1+1+(n+1)+n+1+n(n+1)/2+5\cdot n(n-1)/2$
$= 3n+4$
$= 3+3n+n^2+2n^2-2n$
$= 3n^2+n+3$

(ii) Big-O class: $3n^2+n+3 \rightarrow O(n^2)$

(iii) Best/Worst Case: The best and worst cases are identical because there are no conditional branches. The number of operation only depends on n, not on the contents of arr. Therefore, best case = worst case = $O(n^2)$

b) Snippet B

```
int lo=0, hi=n-1, target=42;
bool found = false;
while (lo <= hi){
    int mid = lo+(hi-lo)/2;
    if (arr[mid] == target) {found = true; break;}
    else if (arr[mid] < target) {lo = mid+1;}
    else {hi = mid-1}
```

- lo <= hi → ⌊log₂ n⌋+2
- mid = lo + (hi-lo)/2 → ⌊log₂ n⌋+1 (executed ~~twice~~ 2x)
- arr[mid] == target → ⌊log₂ n⌋+1
- arr[mid] < target → ⌊log₂ n⌋+1
- lo = mid+1 or hi = mid-1 → ⌊log₂ n⌋+1
- initialization (4) → 4

(i) $T(n) = 4+(\lfloor \log_2 n \rfloor+2+5(\lfloor \log_2 n \rfloor+1)$
$= 6\lfloor \log_2 n \rfloor+11$

(ii) Big-O Class: $6\lfloor \log_2 n \rfloor+11 \rightarrow O(\log n)$

(iii) Best/Worst Case: Different.
- Best case = O(1). When the target is exactly at the midpoint on the first iteration. The loop executes once, find the target, then breaks, which means the cost is constant.
- Worst case = O(log n). When the target is at the leaf of the search, so the loop runs the full ⌊log₂ n⌋+1 iterations before lo>hi terminates the loop.

c) Snippet C

```
int count = 0;
for (int i=0; i<n; i++){
    for (int j=i+1; j<n; j++){
        for (int k=0; k<n; k+=2){
            if (arr[i] != arr[j] && arr[k] > 0)
                count++;
        }
    }
}
```

- count = 0 → 1
- i = 0 → 1
- i<n → n+1
- i++ → n
- j = i+1 → n
- j<n → n(n+1)/2
- j++ → n(n-1)/2
- k = 0 → n(n-1)/2
- k<n → n(n-1)(⌈n/2⌉+1)/2
- k+=2 → n(n-1)⌈n/2⌉/2
- 2 compare if conditions → n(n-1)⌈n/2⌉/2 (~~exceeded~~ 4 operations)
- count++ (conditional) → n(n-1)⌈n/2⌉/2

(i) T(n):
- n is even, ⌈n/2⌉ ≈ n/2 → $T(n) = (2n^3-2n^2)+(2n^2-n)+(4n+1)+2 = 2n^3+3n+3$
- n is odd, ⌈n/2⌉ ≈ (n+1)/2 → $T(n) = (2n^3-2n^2)+(2n^2-n)+(4n+1)+2 = 2n^3+2n+n+3$

(ii) Big-O class: $T(n) = O(n^3)$

(iii) Best/Worst Case: Different, but the Big-O is the same ($O(n^3)$) because the loop structure dominates.
- Best case = $O(n^3)$. For count++: arr[i] == arr[j] for every pair, so the condition never met and count++ never executes.
- Worst case = $O(n^3)$. All elements are distinct and positive, so count++ runs in every innermost iteration.

Problem 3B: Complexity Comparison Table

Operation	Best Case	Worst Case	Average Case
Linear Search (unsorted)	O(1) When the target is the first element	O(n) When the target is the last element or not present	O(n) On average, the target is at the n/2 element, therefore takes n/2 comparison
Linear Search (early exit, sorted)	O(1) When the target is the first element	O(n) When the target is not present and the loop stops when arr[i] > target	O(n) Same as unsorted. Early exit does not help much
Binary Search (sorted)	O(1) When the target is exactly at the midpoint on the first probe	O(log n) When the target is not present or at a leaf. This takes ⌊log₂ n⌋ ~~steps~~ +1 steps.	O(log n) The number of probes is expected to be logarithmic

c) In the worst case, linear search performs n comparisons, while binary search performs $\lfloor \log_2 n \rfloor + 1$ comparisons. We want the smallest n such that: $\lfloor \log_2 n \rfloor + 1 < n$

n	Linear	Binary	Binary faster
1	1	1	No
2	2	2	No
3	3	2	Yes
4	4	3	Yes
10	10	4	Yes
100	100	7	Yes

From the table, we can conclude that binary search becomes strictly more efficient than linear search at ~~n=3~~ minimum $n=3$. For $n>=3$, $\lfloor \log_2 n \rfloor + 1 < n$

Problem 4: Amortized Analysis of pushBack

Part A - Trace

Call #	Value pushed	size after	capacity after	Cost	
1	10	1	2	1	
2	20	2	2	1	
3	30	3	4	3	→ Resize (cap 2→4; cost 1+2=3)
4	40	4	4	1	
5	50	5	8	5	→ Resize (cap 4→8; cost 1+4=5)
6	60	6	8	1	
7	70	7	8	1	
8	80	8	8	1	
9	90	9	16	9	→ Resize (cap 8→16; cost 1+8=9)
10	100	10	16	1	
11	110	11	16	1	
12	120	12	16	1	

Part B - Total Cost

1. Sum of all costs: $T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1 = 26$
2. Average cost per call: $T_{12}/12 = 26/12 = 2.167$
3. Full formula: Resizes occur after pushing elements $2, 4, 8, 16, \ldots, 2^k$. The total copy work across all resizes for n push back is:

$$W(n) = 2+4+8+\ldots+2^{\lfloor \log_2 n \rfloor}$$
$$= \sum_{i=1}^{\lfloor \log_2 n \rfloor} 2^i$$
$$= 2^{\lfloor \log_2 n \rfloor + 1} - 2 \text{ (geometric series with ratio of 2)} < 2n-2$$
$$= O(n)$$

Adding the ~~trivial~~ n trivial write operations: $T(n) = n + W(n) < n + 2n = 3n = O(n)$. This means the amortized cost per push is $O(n)/n = O(1)$ or constant.

Part C - Growth Factor Comparison

1. Trace

Call #	Value pushed	size after	capacity after	Cost	
1	10	1	2	1	
2	20	2	2	1	
3	30	3	6	3	→ Resize (cap 2+4=6; cost 1+2=3)
4	40	4	6	1	
5	50	5	6	1	
6	60	6	6	1	
7	70	7	10	7	→ Resize (cap 6+4=10; cost 1+6=7)
8	80	8	10	1	
9	90	9	10	1	
10	100	10	10	1	
11	110	11	14	11	→ Resize (cap 10+4=14; cost 10+10=11)
12	120	12	14	1	

2. The total cost for this strategy is $T_{12} = 1+1+3+1+1+1+7+1+1+1+1+11+1 = 30$. This is more expensive than the doubling strategy, which takes ~~42~~ 26 cost at 12 calls.
3. T(n) for:
   - Doubling strategy: Copy work: $2+4+8+16+\ldots+2^{\lfloor \log_2 n \rfloor}$
     $W(n) = 2(2^{\lfloor \log_2 n \rfloor - 1})/(2-1) < 2n$
     $T(n) = n + W(n) < 3n = O(n)$
     Amortized cost per call: $O(n)/n = O(1) \rightarrow$ constant
   - Fixed-increment strategy: Copy cost: $2+k(i-1)$
     $W(n) = \sum_{i=1}^{(n-1)/k} 2+k(i-1) \approx n/k \cdot n/2 = n^2/2k \rightarrow O(n^2)$
     $T(n) = n + O(n^2) = O(n^2)$
     Amortized cost per call: $O(n^2)/n = O(n) \rightarrow$ linear, not constant

The doubling strategy achieves $O(1)$ amortized cost per pushBack because the geometric series of resize costs sums to $O(n)$ total. The fixed-increment strategy cost $O(n)$ amortized per pushBack because the arithmetic series of resize costs sums to $O(n^2)$ in total. This shows that doubling (or any constant multiplication factor >1) is preferred for dynamic array growth.